

Contagem paralela de objetos em uma matriz binária

Implementações sequencial e paralela com Pthreads POSIX

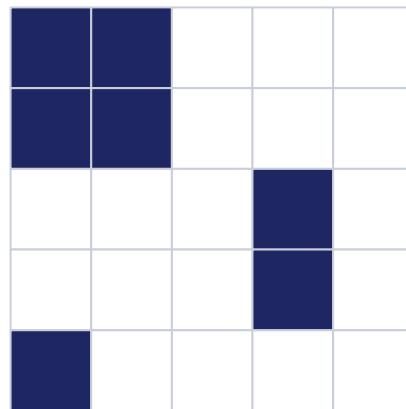
Sistemas Operacionais - 2026/II - Escola Politécnica - PUCRS

Prof. Filipo Novo Mór

Pedro Augusto Wagner · Diogo Giacoboni Gallo

O problema

- A matriz binária é uma imagem: 0 é fundo, 1 é primeiro plano.
- Um objeto é uma região de células 1 ligadas por aresta ou por canto: conectividade 8.
- Contar objetos é contar componentes conexas.
- Duas implementações equivalentes: a sequencial serve de referência de correção, e a paralela precisa dar exatamente o mesmo número.



Exemplo 1 do enunciado: 3 objetos

Estratégia em três fases

1. Rotulagem local

Cada thread recebe uma faixa de linhas e roda o flood fill só dentro dela.

Paralela, sem região crítica.

2. Consolidação

Objetos cortados pelas fronteiras são unificados com union-find.

Paralela, com mutex.

3. Contagem

Conta os rótulos que continuaram sendo raiz da própria classe.

Sequencial, de 1% a 4% do tempo.

Somar contagens locais não funciona: um objeto pode aparecer em duas faixas.

Versão sequencial: a referência

- Varredura da matriz; a primeira célula 1 não visitada inicia um objeto.
- O flood fill marca todas as células alcançáveis pelos 8 vizinhos.
- Pilha explícita em vez de recursão: em 4000 x 4000 a versão recursiva estoura a pilha do processo.
- Cada célula é marcada antes de ser empilhada, então entra na pilha no máximo uma vez: custo $O(\text{linhas} \times \text{colunas})$.

```
objetos++;  
visitado[idx] = 1;  
pilha[topo++] = idx;  
  
while (topo > 0) {  
    atual = pilha[--topo];  
    for (k = 0; k < 8; k++)  
        ... empilha vizinho  
}
```

Decomposição: faixas de linhas

- T faixas contíguas, uma por thread; o resto da divisão vira uma linha a mais nas primeiras faixas.
- Nenhuma thread fica vazia; se T for maior que o número de linhas, T é reduzido.
- Cada thread escreve apenas nas posições das suas linhas: não há dado compartilhado sendo escrito por duas threads.
- Ligação horizontal nunca é cortada por faixas de linhas: as duas células estão sempre na mesma linha.

thread 0 - linhas 0 a 999
thread 1 - linhas 1000 a 1999
thread 2 - linhas 2000 a 2999
thread 3 - linhas 3000 a 3999

3 fronteiras para consolidar

Identificação das componentes locais

- O rótulo de uma componente é o índice linear da sua célula semente somado de 1.
- Como cada célula pertence a uma única faixa, dois rótulos gerados por threads diferentes nunca colidem.
- Consequência: não existe contador global, não existe mutex e não existe região crítica na fase mais cara do programa.
- O vetor de rótulos custa 4 bytes por célula: é o preço de conseguir consolidar as fronteiras depois.

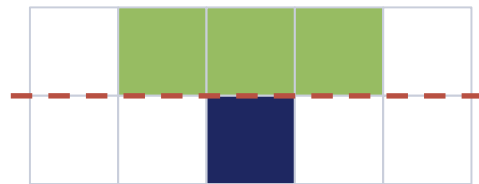
```
rotulo = idx + 1;  
pai[rotulo] = rotulo;  
rotulos[idx] = rotulo;
```

Fase 1 sem sincronização

Escrita disjunta por construção: cada thread só toca as próprias linhas.

Consolidação das fronteiras

- Para cada célula 1 na primeira linha da faixa, olham-se os três vizinhos da última linha da faixa de cima: diagonal esquerda, vertical e diagonal direita.
- Union-find com compressão de caminho; a raiz é sempre o menor rótulo, o que torna o resultado independente da ordem das threads.
- Cada união acontece entre `pthread_mutex_lock` e `pthread_mutex_unlock`: essa é a região crítica.
- Cada thread trata só a sua fronteira superior, então nenhuma fronteira é feita duas vezes.



fronteira entre duas faixas

os três vizinhos cobrem a vertical
e as duas diagonais

Por que Pthreads e não processos

Custo de criação

A medição do material da disciplina: `fork()` custa cerca de 4x mais que `pthread_create()`, porque exige montar uma nova tabela de páginas.

Natureza dos dados

A matriz e o vetor de rótulos são grandes e precisam ser vistos por todos. Com threads eles já são compartilhados.

Com processos seria preciso

`shm_open` mais `mmap`, ou devolver resultados parciais por pipes: mais código, sem mais paralelismo.

Cuidado conhecido

`fork()` depois de criar threads clona mutexes travados. Aqui não há mistura dos dois modelos.

Corretude

Ex.	Dimensões	Esperado	Sequencial	Paralel o
1	5 x 5	3	3	3
2	6 x 8	4	4	4
3	8 x 8	5	5	5
4	9 x 12	6	6	6
5	12 x 12	7	7	7

- Mesmo resultado com 1, 2, 3, 4 e 8 threads.
- Sete testes de borda: matriz vazia, 1 x 1, matriz cheia, diagonal ligada só por cantos, tabuleiro de xadrez, listras horizontais e listras verticais.
- 40 matrizes aleatórias comparadas com a versão sequencial: nenhuma divergência.
- ThreadSanitizer com 4 e 8 threads: nenhum aviso de condição de corrida.

Desempenho: 4000 x 4000 (16 milhões de células)

Versão	Esparsa 30%	Densa 45%
sequencial	0,198 s	0,409 s
1 thread	0,295 s (0,67x)	0,521 s (0,78x)
2 threads	0,166 s (1,20x)	0,278 s (1,47x)
4 threads	0,102 s (1,95x)	0,162 s (2,53x)
8 threads	0,078 s (2,55x)	0,111 s (3,68x)

Mediana de 7 execuções. Máquina: 12 núcleos lógicos, gcc 15.2, WSL2.

- A aceleração cresce de forma monotônica: nenhuma configuração piorou ao receber mais threads.
- A paralela com 1 thread perde da sequencial, e isso não é custo de thread: são os vetores de rótulo e de union-find (144 MB contra 16 MB).
- Descontando esse custo estrutural, o ganho de 1 para 8 threads é de 3,79x na esparsa e 4,69x na densa.
- Matriz 200 x 200: com 8 threads a aceleração cai para 0,58, porque criar as threads custa mais que o trabalho.

Onde o tempo é gasto

Rotulagem (paralela)

0,0965 s
95,5% do tempo

Consolidação (paralela)

0,00053 s
0,5% — única região crítica

Contagem (sequencial)

0,0040 s
4,0% do tempo

- Carga esparsa, 4 threads. A fração sequencial de 4,0% dá um teto de Amdahl de 3,6x com 4 threads e 6,3x com 8.
- Na carga densa há menos objetos, então a contagem final cai para 0,7% e o teto sobe para 7,6x — foi ela que escalou melhor (3,68x).
- Dos 12 núcleos lógicos, metade são de hyper-threading: para uma carga limitada por memória, a segunda thread do mesmo núcleo rende pouco.
- Risco de desequilíbrio: as faixas têm o mesmo número de células, mas não a mesma quantidade de células 1. A correção seria usar mais faixas do que threads, em fila dinâmica.

Conclusões

- A divisão por faixas de linhas resolve o problema com uma única região crítica, e ela custa 0,5% do tempo.
- O rótulo derivado da célula semente elimina o contador compartilhado da fase mais cara.
- Union-find com raiz no menor rótulo torna a resposta determinista, independente da ordem das threads.
- Corretude verificada nas 5 matrizes obrigatórias, em 7 casos de borda e em 40 matrizes aleatórias, de 1 a 8 threads.
- Aceleração de até 3,68x com 8 threads; o paralelismo só compensa em matrizes grandes, e o teto é dado pela fração sequencial e pela banda de memória.